

# Coverage

- Code coverage, expression coverage, etc. are automatically inferred
- Functional coverage specifies what, how and when to gather cover data.
- Normally the objective is to reach 100% coverage for the verification work to be considered complete

# Functional coverage – with triggering event

- `bit [1:0] b;`
- `covergroup cg @(negedge clk);`
- `bcov: coverpoint b;`
- `endgroup`
- `cg cg_inst = new`
- Always sample on every negative clock edge the value of b.
- At the end of the simulation we can get the results in the form: 125 times b was 00, 48 times b was 01, 70 times b was 10, 113 times b was 11.
- If b never was 01, then we have a coverage hole (75% coverage only).
- It's possible to specify a minimal amount of hits for a bin to be considered as covered.

# Functional coverage – no triggering event

```
bit [1:0] b;
```

```
covergroup cg;
```

```
    bcov: coverpoint b;
```

```
endgroup
```

```
cg cg_inst = new; //declare and instantiate a c.group
```

```
initial begin
```

```
    repeat(10) end
```

```
        // do something
```

```
        if (condition)
```

```
            cg_inst.sample();
```

```
        ##1;
```

```
    end
```

```
end
```

# Enabling condition

- **covergroup** cg @(negedge clk);
- bcov: **coverpoint** b **iff** (c==4);
- **endgroup**
- Record coverage on b on every negative clock edge, only if c==4, otherwise ignore (don't sample).
- Can be used in covergroups with or without triggering events
- iff can be used on any coverpoint, on any covergroup.
- Typical usage: deactivate coverage when reset is low

# Automatic bins

- `bit [1:0] b;`
- `enum{a,b,c} letter;`
- **`covergroup`** `cg @(negedge clk);`
- `bcov: coverpoint b;`
- `lettercov: coverpoint letter;`
- **`endgroup`**
- Creates 4 bins for bcov, 3 for lettercov (one for every possible value)
- for 4-valued data types, X and Z are never recorded
- bins are like counters: every time the covergroup is triggered and the variable has a certain value, the bin is incremented by one

# Automatic bins

```
integer i;  
enum{a,b,c} letter;
```

```
covergroup cg @(negedge clk);  
    icov: coverpoint i;  
endgroup
```

- There is a maximal number of automatically-created bins, by default 1024. If the number of values a coverpoint can take is above 1024, the set of all possible values is automatically “sliced”
- In this case 1024 bins will be created – with  $\text{MAXINT}=2^{32}$ :  
[0 , ... ,  $\text{MAXINT}/1024-1$ ]  
[ $\text{MAXINT}/1024$  , ... ,  $2*\text{MAXINT}/1024-1$ ]  
...  
[ $1022*\text{MAXINT}/1024$  , ... ,  $1023*\text{MAXINT}/1024-1$ ]  
[ $1023*\text{MAXINT}/1024$  , ... ,  $\text{MAXINT}-1$ ]

# bins

- It is possible to specify bins instead of using the automatic bins

```
bit [9:0] v_a;
```

```
covergroup cg @(negedge clk);  
  coverpoint v_a  
  {  
    bins a      = { [0:63], 65 }; // values from 0 to 63 or 65  
    bins b[]    = { 200, 201, 202 }; // creates 3 bins  
    bins c      = { [1000:$] }; // from 1000 to 1023  
    bins d[]    = { [10:14], [16:18] }; // creates 7 bins  
    bins others = default; // everything else  
  }  
endgroup
```

a: 1 bin, incremented if v\_a is 65 or is between 0 and 63

b: 3 bins, each is incremented when v\_a takes values 200, 201, 202

c: 1 bin, incremented when v\_a is between 1000 and 1023

d: 7 bins, each is incremented when v\_a takes values 10, 11, 12, 13, 14, 16, 17, 18

others: 1 bin, incremented when v\_a takes any value that is not covered by the other bins

# wildcard bins

- wildcard bins  $a=\{11xx\}$ ;
  - hits for 1100, 1101, 1110 and 1111
- As a symbol for wildcard we can use
  - x
  - z
  - ?



# ignore bins

```
covergroup cg @(negedge clk);  
  coverpoint v_a  
  {  
    ignore_bins ib = {0, 1, 2};  
    bins three = {3};  
    bins four = {4};  
  }  
endgroup
```

- We don't care about when v\_a takes the values 0, 1, 2
- These values are excluded from coverage
- The tool won't signal any lack of coverage if v\_a never was 0, 1 or 2.
- If v\_a never was 3 then bin three is uncovered – coverage hole
- Can be useful for values that the testbench should never generate
- Example: the testbench generates random even numbers, all odd numbers can be defined as ignore\_bins.

# illegal bins

```
covergroup cg @(posedge clk);  
  coverpoint v_a  
  {  
    illegal_bins ib = {0, 1, 2};  
    bins three = {3};  
    bins four = {4};  
  }  
endgroup
```

- If v\_a takes the values 0, 1 or 2 we get a run-time error.
- Can be useful for values that the DUT should never generate
- Example: if the DUT should give in output an even number, all odd numbers can be defined as illegal\_bins. If an odd number is encountered, it means there is a bug in the DUT.

# Cross coverage

```
bit [3:0] a, b;
```

```
covergroup cov @(posedge clk);
```

```
    aXb : cross a, b;
```

```
endgroup
```

- 16x16 automatic bins, one for each combination of values of **a** and **b**
  - how many times a was 0 and b was 0 in the same cycle?
  - how many times a was 0 and b was 1 in the same cycle?
  - ...
  - how many times a was 15 and b was 14 in the same cycle?
  - how many times a was 15 and b was 15 in the same cycle?

# Automatic cross bins

```
enum { red, green, blue } color;
bit [3:0] pixel_adr, pixel_offset, pixel_hue;

covergroup g2 @(posedge clk);
    Hue: coverpoint pixel_hue; // 16 bins
    Offset: coverpoint pixel_offset; // 16 bins
    AxC: cross color, pixel_adr; // 3*16 bins
    all: cross color, Hue, Offset; // 3*16*16 bins
endgroup
```

# Cross coverage

```
bit [31:0] a_var;  
bit [3:0] b_var;  
covergroup cov3 @(posedge clk);  
    A: coverpoint a_var { bins yy[] = { [0:9] }; }  
    CC: cross b_var, A;  
endgroup
```

- cross between one variable and one coverpoint
- 16x10 bins in CC.

# Specifying bins in cross coverage

```
bit [7:0] v_a, v_b;
covergroup cg @(posedge clk);
a: coverpoint v_a
{
  bins a1 = { [0:63] };
  bins a2 = { [64:127] };
  bins a3 = { [128:191] };
  bins a4 = { [192:255] };
}
b: coverpoint v_b
{
  bins b1 = {0};
  bins b2 = { [1:84] };
  bins b3 = { [85:169] };
  bins b4 = { [170:255] };
}
c : cross a, b // 16 bins
{
  bins          c1 = !binsof(a.a4); // 12 bins
  illegal_bins c2 = binsof(a.a2) || binsof(b.b2); // 7 cross products
  ignore_bins  c3 = binsof(a.a1) && binsof(b.b4); // 1 cross product
}
endgroup
```

# Transition bins

- Bins used to specify transitions of values from one sampling time to a future one.
- bins a = (value1 => value2): counts how many times the variable had value1 and in the following sampling time had value2
- bins a = (1 => 3 => 4): var had value 1, then 3, then 4.
- bins a = (1,5 => 6, 7): var had value 1 or 5 and in the next sampling time value 6 or 7 (transitions 1=>6, 1=>7, 5=>6, 5=>7)

# Transition bins

- **bins** sa = (4 => 5 => 6), ([7:9], 10=>11, 12);
- one bin that is incremented for the following transitions:
  - 4=>5=>6
  - 7=>11
  - 8=>11
  - 9=>11
  - 10=>11
  - 7=>12
  - 8=>12
  - 9=>12
  - 10=>12



# Transition bins

- **bins** sa[] = (4 => 5 => 6), ([7:9],10=>11,12);
- 9 bins that are incremented for the following transitions:
  - 4=>5=>6
  - 7=>11
  - 8=>11
  - 9=>11
  - 10=>11
  - 7=>12
  - 8=>12
  - 9=>12
  - 10=>12

# Transition bins

- **wildcard bins**  $T0 = (2'b0x \Rightarrow 2'b1x);$ 
  - increments for
    - $00 \Rightarrow 10;$
    - $01 \Rightarrow 10;$
    - $00 \Rightarrow 11;$
    - $01 \Rightarrow 10;$

# Transition bins support illegal and ignore bins

- **illegal\_bins** bad\_trans = (4=>5=>6);
- **ignore\_bins** bad\_trans = (4=>5=>6);

# Transition bins - Repetition

- $(3 [* 3])$  is equivalent to  $(3 \Rightarrow 3 \Rightarrow 3)$
- Note: if we have four consecutive time 3, then the bin will receive two hits.
- $(3 [*3:5])$  is equivalent to
- $(3 \Rightarrow 3 \Rightarrow 3), (3 \Rightarrow 3 \Rightarrow 3 \Rightarrow 3), (3 \Rightarrow 3 \Rightarrow 3 \Rightarrow 3 \Rightarrow 3)$

# Transition bins - Repetition

- $(2 \Rightarrow 3 [* 3] \Rightarrow 1)$  is equivalent to

$$- (2 \Rightarrow 3 \Rightarrow 3 \Rightarrow 3 \Rightarrow 1)$$

- $(1 \Rightarrow 3 [*3:5])$  is equivalent to

$$(1 \Rightarrow 3 \Rightarrow 3 \Rightarrow 3), (1 \Rightarrow 3 \Rightarrow 3 \Rightarrow 3 \Rightarrow 3), (1 \Rightarrow 3 \Rightarrow 3 \Rightarrow 3 \Rightarrow 3 \Rightarrow 3)$$

# Transition bins - Repetition

- $(1 \Rightarrow 3 [*4:\$] \Rightarrow 2)$  hits every time there is a 1 followed by at least 4 times 3 followed by a 2.

# Goto repetition

- Advanced type of repetition:
- $3 \rightarrow 3$  is equivalent to  $\dots \Rightarrow 3 \Rightarrow \dots \Rightarrow 3 \Rightarrow \dots \Rightarrow 3$
- where  $\dots$  is any transition of any length that does not contain the value 3.
- $1 \Rightarrow 3 \rightarrow 3 \Rightarrow 5$  is equivalent to  
 $1 \Rightarrow \dots \Rightarrow 3 \Rightarrow \dots \Rightarrow 3 \Rightarrow \dots \Rightarrow 3 \Rightarrow 5$